

CODING GUIDELINES FOR CTSYS

Throughout the application we should follow the following coding conventions::

Folder Naming :

Folder Names should be Pascal cased.

Ex:- ErrorLogs, Reports.

File Naming :

File Names should be Pascal cased. In the Model and Controller their names should be followed by a Model/Controller postfix.

Ex:- LoginModel.cfc, LoginController.cfc.

*Use the pre existing file namings.

Variable Naming :

Variable Names should be Camel cased.

Variables should be written with proper scopes as prefix like APPLICATION, SESSION).

Ex:- userInfo(camelCased)

Function Naming :

The functions present inside a component should be named as per their functionality in Pascalcase.

Ex:- We have a function for retrieving data from a table called 'Users', then the function name should be 'getUserDetails'.

White Spaces :

There must be even white space before and after logical operators, syntax should go as it is.

Ex:- var myVar = "ABC";

<cfset myVar="ABC">

There should be one white space between two logical blocks of code.

There must be one white space at a time.

File Header : (put it as a snippet)

```
<!-------
      FileName      : {Filename}
      DateCreated   : {Current Date}
      Description    : {Description}
----->
```

Naming Convention for Stored Procedures :

BACKOFFICE_section_SPNAME

COMMON_SPName

Function Header :

```
/*-----  
    Function Name:  
    Description: This function checks whether the emailId exists or not in the datasource.  
    Arguments: 1. emailId-It describes the emailId.  
               2.id- Its the user id.  
    Return Type: boolean.  
-----*/
```

Inline Comments :

Every critical part of code should be properly commented. It's better to even give inline comments at the start of a block of code accomplishing a particular functionality.

Any file should not be having unused piece of code as commented. If needed then we can comment with proper reasoning.

Use of Hint :

The hint parameter for cfcomponent should be used to provide proper hint.

Ex:- component displayname="Application" output="false" hint="I do pre-page processing for the application".

Try Catch :

Every function statements should be kept inside a try-catch block.

Alert Messages :

Any alert messages, confirm messages, error messages, etc. which are to be displayed to the user, should be in proper tense.

- Suppose, we have a field which is mandatory but the user submits the form without filling value in this field, then the alert message that would be displayed should not have the message something like: "Name field is mandatory!"
- Rather it should have the message something like: "Please provide the name."
- Also it's better to display a red star mark besides the mandatory fields in a form.

Browser Compatibility Testing :

Before uploading or concluding something as done, the application should be tested on different browsers.

The minimum is Mozilla and IE. Before testing rigorously in both the browsers, nothing should be uploaded to the client's system.

Handlers:-

- scope of variables inside function should be local.
- naming of variables should be camelCase.
- naming of functions should be PascalCase.
- CF functions should be in camelCase.
- File Headers should be provided with the details:- File Name, File Description, Created on.
- Functions should have access specifier.
- Functions should have return type as void.
- Functions Header should be there with the following details:- Function Name, Description, Arguments, Return Type.
- No logics in handler functions.
- Maintain proper indentation.
- For a file which would be included in layout provide these mandatory details:- prc.title, prc.innerSection, prc.pageTitle

Action:-

- File Headers should be provided with the details:- File Name, File Description, Created on.
- Functions should have access specifier.
- Functions should have return type as applicable. Avoid "any" if not necessary.
- Functions Header should be there with the following details:- Function Name, Description, Arguments, Return Type.
- Function names should be in PascalCase.
- Variable naming should be in camelCase.
- Proper scope of variables needs to be provided.
- Use Local scope inside functions.
- Scopes should be in all capitals.
- CF Functions should be in camelCase.
- Use try catch block to handle error inside the functions and call APPLICATION.HandleError function in catch block.
- Maintain proper indentation.
- Hint should be provided for every function.

Model:-

- Maintain file header as is previously present.
- Whenever new function are created make sure to use provide function headers.
- Function names should be in Pascalcase.
- Function should have access specifier.
- Function should have a return type.
- Variable naming should be in camelCase.
- Use local scope inside the function.
- Use try catch.
- Hint should be provided.
- Maintain proper indentation.

JS:-

- Maintain or create proper file header with the details:- File Name, File Description, DateCreated.
- Every function should have proper function header with the details:- Function Name, Description.
- Function naming should be in PascalCase.
- Remove any unwanted/commented out code which is not necessary.
- All the js codes needs to be in the js file and should not be in cfm pages.
- The js files must be included at the end of the page.
- Variable names should be in camelCase.

cfm:-

- Proper file header with the details:- File Name, File Description, DateCreated.
- ID should be in PascalCase.
- dev created classes should be in camelCase.
- Proper indentation is a must.
- Include the css file at the start of the page.
- Include the js file at the end of the page.
- Donot use cf display tags in the page. For ex:- cfform, cfinput, cfgrid, cfmessagebox, cfwindow.

css:-

- Avoid inline css.
- Create classes and use them whenever needed.
- Create one css file which will be used site wide.

For Alerts:-

Use Notification function for any type of alert.

Notification(title, text, type, callback, value)

title:- The heading of the alert box.

text:- The message of the alert box.

type:- The type of alert box (alert, warning, input/prompt, success, error)

callback:- The function name only should be provided which will be called on clicking "OK".

data:- If any data needs to be passed to the function this can be done by sending it here.

If multiple data needs to be provided then create global variables store the data inside those and on the callback function use them as necessary.

DONOT do :-

Notification("Company", "Successfully registered", "success", AddCompanyCallback(id, msg));

It should be always like this:-

Notification("Company", "Successfully registered", "success", AddCompanyCallback);

For Modal window:-

Use the function

`ModalWindow(id, title, url, callback)`

Id:- The id of the modal window which is now GeneralWindow.

title:- The heading of the modal window.

url:- The URL of the page which would be loaded in the modal window.

callback:- If a callback is needed then you can pass a callback function name here.